



FACULTAD DE INGENIERÍA - UNIVERSIDAD DE BUENOS AIRES
TB024 — TEORÍA DE ALGORITMOS
CÁTEDRA PODBEREZSKI

Trabajo Práctico N.º 1

Grupo: TDA CODE

Entrega: 1er Entrega

Fecha de Entrega: 21 de Septiembre de 2026

Apellido y Nombre	Padrón
Quino, Julian	94224
Martinez, Juan	94785
Martinez, Matias	113387
Vasquez, Lucas	112435
Rodriguez, Nazareno Jose Luis	101891

Índice

1. Parte 1: Armado de lavarropas	3
1.1. Punto 1: Solución propuesta, funciones de costo y límite, y criterios de poda	3
1.1.1. Análisis y Planteo del Problema	3
1.1.2. Estrategia de Resolución	3
1.1.3. Criterios de Poda	3
1.2. Punto 2: Pseudocódigo y estructuras de datos	4
1.2.1. Pseudocódigo del algoritmo	4
1.2.2. Estructuras de datos	4
1.3. Punto 3: Análisis de complejidad temporal y espacial	5
1.3.1. Complejidad Temporal	5
1.3.2. Complejidad Espacial	5
1.4. Punto 4: Ejemplo paso a paso	5
1.5. Punto 5: Implementación	6
2. Parte 2 : Los invictos del torneo (División y conquista)	7
2.1. Estrategia	7
2.1.1. Pre-Ordenar	7
2.1.2. Dividir y Conquistas	7
2.1.3. Combinación	7
2.2. Relación de recurrencia	7
2.3. Pseudocódigo	8
2.4. Análisis de Complejidad	9
2.5. Respuestas	9
2.5.1. Pregunta 1	9
2.5.2. Pregunta 2	9
3. Parte 3: El escritorio del gran copista	10
3.1. Estrategia Greedy y criterio de elección	10
3.2. Criterios Greedy incorrectos y contra ejemplos	10
3.2.1. Criterio Incorrecto 1: Mayor importancia primero (w_i descendente)	10
3.2.2. Criterio Incorrecto 2: Menor tiempo de copiado primero (t_i ascendente)	10
3.3. Conceptos teóricos de Algoritmos Codiciosos	11
3.4. Pseudocódigo	11
3.5. Demostración formal de optimalidad (Argumento de Intercambio)	12
4. Análisis de Complejidad	12
4.1. Rol del tiempo fijo de entrega a_i	12
4.2. Ejemplo paso a paso	12

1. Parte 1: Armado de lavarropas

1.1. Punto 1: Solución propuesta, funciones de costo y límite, y criterios de poda

1.1.1. Análisis y Planteo del Problema

El problema consiste en subdividir un conjunto de n prendas de vestir en la menor cantidad posible de lavarropas de capacidad ilimitada, respetando un conjunto de m incompatibilidades entre prendas.

Podemos modelar este problema de forma directa mediante la teoría de grafos:

- **Vértices (V):** representan cada una de las prendas a lavar, identificadas por un número entero del 1 al n .
- **Aristas (E):** representan las incompatibilidades existentes entre dos prendas (es decir, si existe la arista (u, v) , las prendas u y v no pueden compartir el mismo lavarropas).
- **Colores:** corresponden a las etiquetas asociadas a cada lavarropas (e.g. "A", "B", "C" ... o números enteros $1, 2, \dots, k$).

De este modo, el objetivo del algoritmo es encontrar el **Número Cromático** $\chi(G)$ del grafo de incompatibilidades, el cual indica la mínima cantidad de lavarropas necesarios para asignar a todas las prendas de forma válida.

1.1.2. Estrategia de Resolución

La estrategia propuesta se basa en la técnica de **Backtracking combinada con Branch and Bound**, aplicando un algoritmo de búsqueda en profundidad (**DFS**).

La idea es construir la solución asignando lavarropas prenda por prenda:

- **Árbol de Estados:** Cada nivel i del árbol (con i desde 1 hasta n) corresponde a la decisión de asignar un lavarropas a la prenda i .
- **Función Límite (EsAsignacionValida):** Para la prenda actual i y un lavarropas candidato c , verifica que ninguna prenda vecina (incompatible) ya asignada se encuentre usando el mismo lavarropas c .
- **Función Costo / Cota:** Se mantiene la variable `cant_lavarropas_rama` en la rama en curso y una variable global `min_cantidad_lavarropas_necesarios_hallada` que almacena el récord de la mejor solución completa obtenida hasta el momento.

1.1.3. Criterios de Poda

El algoritmo aplica tres criterios de poda para evitar exploraciones innecesarias y reducir el espacio de búsqueda:

1. **Poda por Inviabilidad:** Si la prenda actual no puede ser colocada en el lavarropas candidato c porque entraría en conflicto con alguna prenda vecina ya asignada, se descarta esa rama de inmediato.
2. **Poda por Acotamiento (Branch and Bound):** Si en algún punto del recorrido la cantidad de lavarropas abiertos iguala o supera el mejor resultado encontrado hasta el momento (`cant_lavarropas_rama` $\geq$ `min_cantidad_lavarropas_necesarios_hallada`), se cierra el seguimiento de esa rama. Esto se justifica porque continuar asignando las prendas restantes nunca permitirá obtener una solución con menos lavarropas que el récord ya registrado.
3. **Poda por Simetría de Etiquetas:** Para la prenda actual, solo se evalúan los lavarropas ya abiertos hasta el momento y a lo sumo **un único lavarropas nuevo** (`cant_lavarropas_rama` + 1). Esto evita probar etiquetas simétricas para lavarropas vacíos que representarían particiones físicamente idénticas.

1.2. Punto 2: Pseudocódigo y estructuras de datos

1.2.1. Pseudocódigo del algoritmo

```
1 FUNCION ArmarLavarropas(cantidad_prendas, incompatibilidades)
2   grafo <- ConstruirGrafoAdyacencia(cantidad_prendas, incompatibilidades)
3   prendas_sorted <- OrdenarPrendasPorGradoDecreciente(grafo)
4
5   asignacion <- Arreglo[cantidad_prendas + 1] con valor_inicial = 0
6   min_cantidad_lavarropas_necesarios_hallada <- cantidad_prendas + 1
7   mejor_asignacion <- NULO
8
9   FUNCION BacktrackingLavarropas(idx, cant_lavarropas_rama)
10    // Poda por Acotamiento
11    SI cant_lavarropas_rama >= min_cantidad_lavarropas_necesarios_hallada ENTONCES
12      RETORNAR
13    FIN SI
14
15    // Caso Base: Se asignaron todas las prendas
16    SI idx > cantidad_prendas ENTONCES
17      min_cantidad_lavarropas_necesarios_hallada <- cant_lavarropas_rama
18      mejor_asignacion <- COPIAR(asignacion)
19      RETORNAR
20    FIN SI
21
22    prenda <- prendas_sorted[idx]
23
24    // Poda por Simetria: Probar lavarropas usados + 1 nuevo
25    PARA lavarropas DESDE 1 HASTA (cant_lavarropas_rama + 1)
26      // Poda por Inviabilidad
27      SI EsAsignacionValida(prenda, lavarropas, grafo, asignacion) ENTONCES
28        asignacion[prenda] <- lavarropas
29
30        BacktrackingLavarropas(idx + 1, MAXIMO(cant_lavarropas_rama, lavarropas))
31
32        asignacion[prenda] <- 0 // Backtrack
33      FIN SI
34    FIN PARA
35  FIN FUNCION
36
37  BacktrackingLavarropas(idx = 1, cant_lavarropas_rama = 0)
38  RETORNAR MapearEtiquetas(mejor_asignacion)
39 FIN FUNCION
40
41 FUNCION EsAsignacionValida(prenda, lavarropas, grafo, asignacion)
42   PARA CADA vecina EN grafo[prenda]
43     SI asignacion[vecina] == lavarropas ENTONCES
44       RETORNAR FALSO
45     FIN SI
46   FIN PARA
47   RETORNAR VERDADERO
48 FIN FUNCION
```

Listing 1: Pseudocódigo de la solución propuesta para el armado de lavarropas

1.2.2. Estructuras de datos

- **Grafo de Incompatibilidades (grafo):** Representado mediante una lista de adyacencia basada en conjuntos (set), lo que permite consultar la existencia de vecinos en tiempo constante promedio $O(1)$.
- **Arreglo de Asignación (asignacion):** Arreglo indexado donde la posición i almacena el número de lavarropas asignado a la prenda i .
- **Arreglo de Ordenamiento (prendas_sorted):** Lista con las prendas ordenadas de mayor a menor según su grado de incompatibilidad (Heurística Welsh-Powell), asegurando procesar primero las prendas con más restricciones para acotar el árbol más rápido.

1.3. Punto 3: Análisis de complejidad temporal y espacial

1.3.1. Complejidad Temporal

■ Fase de Preprocesamiento:

- *Construcción del Grafo:* Leer las m aristas del archivo e insertarlas en los conjuntos de adyacencia (**set**) requiere tiempo $\mathcal{O}(n + m)$, dado que la inserción en tablas hash toma $\mathcal{O}(1)$ promedio por arista.
- *Cálculo de Grados y Ordenamiento:* Obtener el grado de incompatibilidad de las n prendas toma $\mathcal{O}(n)$, ya que consultar el tamaño del conjunto (**len**) es una operación $\mathcal{O}(1)$. Luego, la ordenación decreciente de los n vértices según su grado (heurística de Welsh-Powell) se realiza invocando el algoritmo **Timsort** el cual garantiza una cota superior de complejidad de $\mathcal{O}(n \log n)$ en el peor caso.

- **Fase de Búsqueda por Backtracking con Branch and Bound:** En el peor caso teórico (por ejemplo, en un grafo nulo o de baja densidad sin podas efectivas), el árbol de estados completo de profundidad n con un alfabeto de k lavarropas posee $\mathcal{O}(k^n)$ estados. En cada estado parcial, la función de validación (**EsAsignacionValida**) verifica el conflicto de color con las prendas adyacentes evaluando a lo sumo $\mathcal{O}(\text{grado}(v)) \leq \mathcal{O}(n)$ vecinos.

Siguiendo la formulación teórica dada en los apuntes de la cátedra [1], el trabajo por nodo multiplicado por la cantidad total de estados del árbol resulta en una cota superior en el peor caso de:

$$\mathcal{O}(n \cdot k^{n+1}) \quad \text{o simplemente } \mathcal{O}(n \cdot k^n)$$

Al aplicar la Poda por Simetría de etiquetas, el número de estados a explorar se reduce de $\mathcal{O}(k^n)$ al número de particiones de conjunto dado por los números de Bell B_n , obteniendo una cota ajustada de $\mathcal{O}(n \cdot B_n)$.

- **Complejidad Temporal Global:** La complejidad temporal total del algoritmo se expresa formalmente como:

$$T(n, m) = \mathcal{O}(n \log n + m) + \mathcal{O}(T_{\text{búsqueda}}(n))$$

En el análisis asintótico de peor caso ($n \rightarrow \infty$), el término exponencial de la búsqueda domina la cota superior del algoritmo. Sin embargo, el preprocesamiento de ordenamiento inicial por grados, sumado al efecto combinado de las podas por inviabilidad, acotamiento y simetría de etiquetas, reduce la cantidad efectiva de nodos explorados en el árbol de estados a una fracción acotada del espacio combinatorial para las instancias del problema.

1.3.2. Complejidad Espacial

- **Grafo de Incompatibilidades:** La estructura de adyacencia requiere un espacio de $\mathcal{O}(n + m)$, siendo n la cantidad de prendas y m la cantidad de relaciones incompatibles.
- **Memoria de Trabajo y Pila DFS:** La profundidad máxima de la pila de recursión DFS no supera la cantidad total de prendas n , lo que requiere $\mathcal{O}(n)$ espacio.
- **Complejidad Espacial Total:** Resulta en $\mathcal{O}(n + m)$.

1.4. Punto 4: Ejemplo paso a paso

Se presenta a continuación el seguimiento detallado de la ejecución del algoritmo para el archivo de entrada del ejemplo:

p 5 3 | Incompatibilidades: (1, 3), (1, 2), (1, 4)

1. Primera decisión (Prenda 1):

- Se selecciona la prenda 1 por ser la de mayor grado (3 incompatibilidades).
- Se asigna al **Lavarropas 1**.

- Estado: `cant_lavarropas_rama = 1`.

2. Segunda decisión (Prenda 2):

- Se evalúa Lavarropas 1: falla por incompatibilidad con la prenda 1.
- Se evalúa el nuevo Lavarropas 2: se cumple la asignación.
- Estado: `cant_lavarropas_rama = 2`.

3. Tercera decisión (Prenda 3):

- Se evalúa Lavarropas 1: falla por incompatibilidad con la prenda 1.
- Se evalúa Lavarropas 2: se cumple (la prenda 3 no es incompatible con la prenda 2).
- Estado: `cant_lavarropas_rama = 2`.

4. Cuarta decisión (Prenda 4):

- Se evalúa Lavarropas 1: falla por incompatibilidad con la prenda 1.
- Se evalúa Lavarropas 2: se cumple (la prenda 4 no es incompatible con 2 ni con 3).
- Estado: `cant_lavarropas_rama = 2`.

5. Quinta decisión (Prenda 5):

- No posee incompatibilidades registradas.
- Se evalúa Lavarropas 1: se cumple la asignación.
- Estado: `cant_lavarropas_rama = 2`.

6. Resultado final:

- Se alcanza el caso base con las 5 prendas asignadas.
- Se actualiza el resultado óptimo obteniendo `min_cantidad_lavarropas_necesarios_hallada = 2`.
- Salida generada: [(1, ".^"), (2, "B"), (3, "B"), (4, "B"), (5, ".^")].

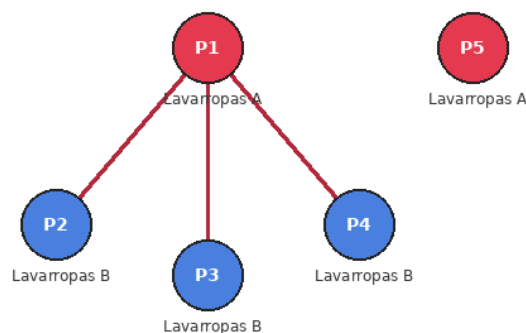


Figura 1: Grafo de incompatibilidades coloreado

1.5. Punto 5: Implementación

El código fuente asociado a esta solución se encuentra disponible en el archivo `tp1_1.py`. La función principal `main(filepath)` se encarga de parsear el archivo de entrada, construir la lista de adyacencia y ejecutar el algoritmo de Backtracking con Branch and Bound, devolviendo la lista de tuplas requerida. La solución fue comprobada mediante la suite de pruebas unitarias provista en `tests.py`.

2. Parte 2 : Los invictos del torneo (División y conquista)

2.1. Estrategia

2.1.1. Pre-Ordenar

Primero ordenamos a las criaturas con un método de ordenamiento $\mathcal{O}(n \log n)$. El orden debe realizarse de forma creciente con el ataque, pero de forma decreciente en defensa. Esto por la regla del torneo exige una ventaja estricta en ataque. Si dos criaturas tienen el mismo ataque, ninguna puede dominar a la otra. Al poner la de mayor defensa a la izquierda, evitas que en la combinación la de la derecha intente eliminar por error a la de la izquierda.

2.1.2. Dividir y Conquistas

Dividir el vector de criaturas en subvectores hasta quedar con una criatura, ya que es el caso trivial, una criatura no se puede dominar a sí misma. Esto se logra haciendo llamados recursivos pasando como parámetro los subvectores izquierdo y derecho.

2.1.3. Combinación

Antes de Combinar se obtiene la máxima defensa del lado derecho y el mínimo ataque del lado derecho. Para luego limpiar la parte izquierda, ya que al estar ordenado en ataque, las criaturas de la derecha pasan a la solución del nivel no porque sean las más fuertes en general, sino porque las de la izquierda tienen menor ataque. Por ello las criaturas de la izquierda dependen de tener una defensa mayor a la máxima de las de la derecha o poseer un ataque igual a la primera criatura de la derecha y una defensa mayor o igual, para no ser dominadas. Siendo el último un caso particular donde hay duplicados.

2.2. Relación de recurrencia

División: Se calcula el punto medio para separar el conjunto en dos subproblemas de tamaño $n/2$. Esto toma un tiempo constante $\mathcal{O}(1)$.

Conquista: Se resuelve de forma recursiva, haciendo 2 llamadas para dividir el subproblema en un tamaño de $n/2$. Teniendo un costo de $2T(n/2)$.

Combinación: Se busca la máxima defensa de las criaturas del lado derecho lo que requiere un tiempo de $\mathcal{O}(n/2)$. Además se busca el ataque de la primera criatura que es $\mathcal{O}(1)$. Luego se usa ese máximo para filtrar a las criaturas del lado izquierdo lo cual da un costo de $\mathcal{O}(n/2)$. Por ende el trabajo no recursivo de combinar es lineal $\mathcal{O}(n)$.

Ecuación de recurrencia

$$T(n) = 2T(n/2) + f(n)$$

Aplicar Teorema Maestro

Calcular el límite $n^{\log_b a} = n^{\log_2 2} = n^1 = n$.

Comparó $f(n)$ con $n^{\log_2 2}$. Lo cual me pone en el caso 2 del teorema maestro.

Por lo tanto la complejidad de la función recursiva es $T(n) = \mathcal{O}(n \log n)$.

2.3. Pseudocódigo

```
1 ordenar_criaturas(criaturas)
2   cantidad = longitud(criatura)
3   Si cantidad <= 1 entonces
4     devolver criaturas
5
6   medio = cantidad / 2
7   izquierda = subarreglo(criaturas, 0, medio)
8   derecha = subarreglo(criaturas, medio, cantidad)
9   izquierda_ordenada = ordenar_criaturas(izquierda)
10  derecha_ordenada = ordenar_criaturas(derecha)
11  devolver merge(izquierda_ordenada, derecha_ordenada)
12
13 merge(izquierda, derecha)
14   ordenados = lista_vacia()
15   i = 0
16   j = 0
17   mientras i < longitud(izquierda) Y j < longitud(derecha) hacer
18     si izquierda[i].ataque < derecha[j].ataque entonces
19       agregar izquierda[i] a ordenados
20       i = i + 1
21     sino si izquierda[i].ataque > derecha[j].ataque entonces
22       agregar derecha[j] a ordenados
23       j = j + 1
24     sino
25       si izquierda[i].defensa >= derecha[j].defensa entonces
26         agregar izquierda[i] a ordenados
27         i = i + 1
28       sino
29         agregar derecha[j] a ordenados
30         j = j + 1
31   mientras i < longitud(izquierda) hacer
32     agregar izquierda[i] a ordenados
33     i = i + 1
34   mientras j < longitud(derecha) hacer
35     agregar derecha[j] a ordenados
36     j = j + 1
37   devolver ordenados
38
39 resolver_combates(criaturas)
40   cantidad = longitud(criaturas)
41   si cantidad <= 1 entonces
42     devolver criaturas
43   mitad = cantidad / 2
44
45   criaturas_izquierda = subarreglo(criaturas, 0, mitad)
46   criaturas_derecha = subarreglo(criaturas, mitad, cantidad)
47
48   criaturas_izquierda_invictas = resolver_combate(criaturas_izquierda)
49   criaturas_derecha_invictas = resolver_combates(criaturas_derecha)
50   devolver combinacion_soluciones(criaturas_izquierda_invictas,
51   criaturas_derecha_invictas)
52
53 combinacion_soluciones(criaturas_izquierda_invictas, criaturas_derecha_invictas)
54   si criaturas_izquierda_invictas esta vacia entonces
55     devolver criaturas_derecha_invictas
56   si criaturas_derecha_invictas esta vacia entonces
57     devolver criaturas_izquierda_invictas
58
59   atk_min = criaturas_derecha_invictas[0].ataque
60   def_max = maxima_defensa(criaturas_invictas_derecha)
61   criaturas_izquierda_filtradas = lista_vacia()
62
63   para cada criatura en criaturas_izquierda_invictas hacer
64     si criatura.defensa > def_max O (criatura.defensa >= def_max Y criatura.ataque ==
65     atk_min) entonces
66       agregar criatura a criaturas_izquierda_filtradas
67
68   devolver concatenar(criaturas_izquierda_filtradas, derecha)
```

2.4. Análisis de Complejidad

La complejidad del método de ordenamiento, en este caso mergesort, es de complejidad temporal de $\mathcal{O}(n \log n)$ y una complejidad espacial de $\mathcal{O}(n)$.

Como ya se calculó antes la complejidad del algoritmo para resolver el problema de criaturas invictas tiene una complejidad $\mathcal{O}(n \log n)$ y una complejidad espacial de $\mathcal{O}(n)$.

En total la complejidad resolutive temporal es $\mathcal{O}(n \log n)$, y es necesario el pre-ordenamiento con complejidad, para que no aumente la complejidad. Ya que habría que enfrentar a cada criatura de la izquierda una por una con todas las criaturas de la derecha. Lo cual daría una complejidad $\mathcal{O}(n^2)$.

2.5. Respuestas

2.5.1. Pregunta 1

En el peor de los casos, ninguna criatura es dominada por otra y las N criaturas forman parte del conjunto de invictos. Para que esto fuera posible la configuración sería $\{(1, N), (2, N - 1) \dots (N, 1)\}$. Donde cada criatura supera las posteriores en ataque, pero tiene menor defensa, por lo que ninguna logra dominar a otra. Esto implica que la salida alcanza su cota superior de N criaturas. No implica ningún cambio con respecto a la complejidad espacial o temporal.

2.5.2. Pregunta 2

Si comparten los mismos valores de ataque y defensa todas son invictas ya que para dominar tiene que tener mayor ataque o igual o mayor defensa. Si comparten el mismo poder de ataque, igualmente ambas siguen siendo invictas ya que es requisito para dominar que tenga mayor ataque y mayor o igual defensa.

El algoritmo pide el ataque de la primera criatura del lado derecho, para filtrar a las del lado izquierdo, si hay criaturas con mismo ataque deben ser invictas ya que del lado derecho hay criaturas invictas que no fueron dominadas. Aclaro que solo si no hay criatura que pueda dominarlo.

3. Parte 3: El escritorio del gran copista

3.1. Estrategia Greedy y criterio de elección

La estrategia greedy consiste en ordenar los pergaminos de forma decreciente según el cociente w_i/t_i . De esta manera, se priorizan los pergaminos que aportan una mayor importancia por unidad de tiempo de copiado.

Este criterio resulta intuitivo porque permite completar antes aquellos pergaminos cuya relevancia es alta en comparación con el esfuerzo requerido para copiarlos. Al reducir el tiempo de finalización de estos pergaminos, también disminuye su contribución a la molestia total.

3.2. Criterios Greedy incorrectos y contra ejemplos

3.2.1. Criterio Incorrecto 1: Mayor importancia primero (w_i descendente)

- **Propuesta:** Priorizar el copiado de los pergaminos ordenándolos de mayor a menor según su peso de importancia (w_i), pensando que resolver antes lo más urgente reduce la molestia total.
- **Contra ejemplo:** Consideremos dos pergaminos con los siguientes datos:
 - Pergamino A: tarda 100 horas en copiarse ($t_A = 100$) y tiene importancia 10 ($w_A = 10$).
 - Pergamino B: tarda 1 hora en copiarse ($t_B = 1$) y tiene importancia 9 ($w_B = 9$).
- **Demostración de fallo:**
 - **Según la regla propuesta (hacer primero A y luego B):** Como A es un poco más importante ($10 > 9$), se copia primero. El pergamino A termina en la hora 100 con una molestia de $10 \cdot 100 = 1000$. El pergamino B debe esperar y termina en la hora 101, generando una molestia de $9 \cdot 101 = 909$. La molestia total sumada es de **1909**.
 - **Probando el orden contrario (hacer primero B y luego A):** El pergamino B termina en la hora 1 con una molestia de $9 \cdot 1 = 9$. Luego A termina en la hora 101 con una molestia de $10 \cdot 101 = 1010$. La molestia total sumada baja a **1019**.

El criterio falla porque se fija solo en la importancia e ignora que hacer un pergamino extremadamente largo de primero hace esperar a todo el resto, disparando la molestia general.

3.2.2. Criterio Incorrecto 2: Menor tiempo de copiado primero (t_i ascendente)

- **Propuesta:** Darle prioridad a los pergaminos que tardan menos tiempo en copiarse (t_i), pensando que al sacarse de encima rápido la mayor cantidad de tareas posibles se reduce la espera general.
- **Contra ejemplo:** Imaginemos dos pergaminos con los siguientes datos:
 - Pergamino A: tarda 1 hora en copiarse ($t_A = 1$), pero es poco importante ($w_A = 1$).
 - Pergamino B: tarda 2 horas en copiarse ($t_B = 2$), pero es extremadamente importante ($w_B = 100$).
- **Demostración de fallo:**
 - **Según la regla propuesta (hacer primero A y luego B):** El pergamino A termina en la hora 1 y genera una molestia de $1 \cdot 1 = 1$. Luego, el pergamino B termina en la hora 3 ($1 + 2$) y genera una molestia de $100 \cdot 3 = 300$. La molestia total sumada es de **301**.
 - **Probando el orden contrario (hacer primero B y luego A):** El pergamino B termina en la hora 2 y genera una molestia de $100 \cdot 2 = 200$. Después, el pergamino A termina en la hora 3 ($2 + 1$) y genera una molestia de $1 \cdot 3 = 3$. La molestia total sumada baja a **203**.

Este criterio falla porque se enfoca únicamente en terminar rápido las tareas cortas, dejando esperando a pergaminos críticos cuya demora cuesta muchísimo más.

3.3. Conceptos teóricos de Algoritmos Codiciosos

- **Elección Greedy:** En cada paso se selecciona el pergamino con mayor cociente w_i/t_i entre los que aún no fueron copiados.
- **Óptimo Local vs. Óptimo Global:** La elección local consiste en copiar primero el pergamino con mayor relación importancia-tiempo. Se demuestra que realizar esta elección en cada paso conduce al mínimo coste total global.
- **Subestructura Óptima:** Una vez elegido el primer pergamino según el criterio w_i/t_i , el problema restante conserva la misma estructura: encontrar el mejor orden para los pergaminos restantes.

3.4. Pseudocódigo

```
1 FUNCION OrdenarPergaminosOptimo ( pergaminos )
2   // Calculamos el ndice de cada pergamino
3   PARA CADA pergamino EN pergaminos
4     pergamino.indice <- pergamino.w / pergamino.t
5   FIN PARA
6   // Ordenamos de mayor a menor ndice
7   pergaminos_ordenados <- OrdenarPorIndice ( pergaminos )
8   RETORNAR pergaminos_ordenados
9 FIN FUNCION
10
11
12 FUNCION OrdenarPorIndice ( pergaminos )
13   SI cantidad de pergaminos <= 1 ENTONCES
14     RETORNAR pergaminos
15   FIN SI
16
17   // Dividimos los pergaminos en dos mitades
18   mitad <- cantidad de pergaminos / 2
19
20   izquierda <- primera mitad de pergaminos
21   derecha <- segunda mitad de pergaminos
22
23   izquierda <- OrdenarPorIndice ( izquierda )
24   derecha <- OrdenarPorIndice ( derecha )
25   resultado <- arreglo [ cantidad de pergaminos ]
26
27   pos_izquierda <- 1
28   pos_derecha <- 1
29   posicion <- 1
30
31   // Combinamos las mitades de mayor a menor ndice
32   MIENTRAS pos_izquierda <= cantidad de izquierda Y pos_derecha <= cantidad de derecha
33     SI izquierda[pos_izquierda].indice >= derecha[pos_derecha].indice ENTONCES
34       resultado[posicion] <- izquierda[pos_izquierda]
35       pos_izquierda <- pos_izquierda + 1
36     SINO
37       resultado[posicion] <- derecha[pos_derecha]
38       pos_derecha <- pos_derecha + 1
39     FIN SI
40     posicion <- posicion + 1
41   FIN MIENTRAS
42
43   MIENTRAS pos_izquierda <= cantidad de izquierda
44     resultado[posicion] <- izquierda[pos_izquierda]
45     pos_izquierda <- pos_izquierda + 1
46     posicion <- posicion + 1
47   FIN MIENTRAS
48
49   MIENTRAS pos_derecha <= cantidad de derecha
50     resultado[posicion] <- derecha[pos_derecha]
51     pos_derecha <- pos_derecha + 1
52     posicion <- posicion + 1
53   FIN MIENTRAS
54   RETORNAR resultado
```

Listing 2: Pseudocódigo de ordenamiento Greedy para copiado de pergaminos

3.5. Demostración formal de optimalidad (Argumento de Intercambio)

Probaremos por **contradicción** que la regla greedy siempre logra el costo mínimo.

Nuestra regla establece que un pergamino A va justo antes que B si cumple que $w_A \cdot t_B \geq w_B \cdot t_A$. Supongamos que el algoritmo **no es óptimo**. Entonces tendría que existir una lista óptima S^* con al menos dos pergaminos adyacentes, A y B , colocados al revés de nuestra regla:

$$w_A \cdot t_B < w_B \cdot t_A$$

Si creamos una nueva lista S' intercambiando el orden de esa pareja (haciendo primero B y luego A), no alteramos el tiempo de finalización de ningún otro pergamino antes o después de ellos. La diferencia en la molestia total entre ambas listas se reduce a evaluar el impacto de A y B :

$$\text{Diferencia de molestia} = f(S') - f(S^*) = w_A \cdot t_B - w_B \cdot t_A$$

Como por hipótesis teníamos que $w_A \cdot t_B < w_B \cdot t_A$, la diferencia resulta ser estrictamente negativa:

$$\text{Diferencia} < 0 \implies f(S') < f(S^*)$$

Esto demuestra que al intercambiarlos bajamos la molestia total, lo cual ****contradice**** la idea de que S^* ya era la mejor lista posible. Por ende, el ordenamiento *greedy* es garantizadamente óptimo.

4. Análisis de Complejidad

- **Complejidad Temporal:** $O(n \log n)$, sería porque usamos el algoritmo de ordenamiento por división y conquista (*Merge Sort*) que utilizamos para ordenar los índices de los pergaminos.
- **Complejidad Espacial:** $O(n)$, necesaria para guardar los arreglos auxiliares al combinar las mitades durante el ordenamiento.

4.1. Rol del tiempo fijo de entrega a_i

Si lo piensas en voz alta, el tiempo que tarda el mensajero en llevar un pergamino es algo que se va a sumar sí o sí al final, sin importar en qué orden los copie el copista.

Matemáticamente, si abrimos la suma de la molestia total nos queda por un lado la molestia del trabajo del copista y por el otro la molestia acumulada por los envíos:

$$\sum w_i \cdot (C_i + a_i) = \sum w_i \cdot C_i + \sum w_i \cdot a_i$$

Como cada pergamino i ya viene con su importancia w_i y su tiempo de entrega a_i fijos desde la mañana, la suma de los productos de $w_i \cdot a_i$ siempre va a dar exactamente el mismo número sin importar cómo los ordenemos. Es una constante fija. Por eso, para minimizar la molestia total del reino, la única parte que realmente podemos controlar y que debemos minimizar es la suma de $w_i \cdot C_i$. El tiempo de entrega simplemente no influye en la decisión.

4.2. Ejemplo paso a paso

Imaginemos que la mañana empieza y el copista recibe 3 pergaminos para trabajar:

- **Pergamino 1:** Tarda 2 horas y tiene importancia 10.
- **Pergamino 2:** Tarda 1 hora y tiene importancia 1.
- **Pergamino 3:** Tarda 3 horas y tiene importancia 9.

Paso 1: Calcular la razón para cada pergamino

El algoritmo calcula el índice dividiendo la importancia entre el tiempo:

- Pergamino 1: $10/2 = 5$
- Pergamino 2: $1/1 = 1$
- Pergamino 3: $9/3 = 3$

Paso 2: Ordenar de mayor a menor

El algoritmo ordena los pergaminos guiándose por este número. Como 5 es mayor que 3, y 3 es mayor que 1, la secuencia elegida para trabajar será: ****Pergamino 1, luego Pergamino 3, y al final Pergamino 2****.

Paso 3: Ver el resultado

Siguiendo este orden:

- El Pergamino 1 se hace primero, termina en la hora 2 y genera una molestia de $10 \cdot 2 = 20$.
- El Pergamino 3 se hace a continuación, termina en la hora 5 ($2 + 3$) y genera una molestia de $9 \cdot 5 = 45$.
- El Pergamino 2 se hace al final, termina en la hora 6 ($5 + 1$) y genera una molestia de $1 \cdot 6 = 6$.

Sumando todo, la molestia total del trabajo es $20 + 45 + 6 = 71$. Cualquier otro orden de copiado que se intente dará una molestia mayor, demostrando cómo funciona el algoritmo en la práctica.

Referencias

- [1] Podberezski, V. D. *Apunte TDA - Backtracking y branch and bound*. Teoría de Algoritmos (TB024), Facultad de Ingeniería, Universidad de Buenos Aires (FIUBA).
- [2] Kleinberg, J., & Tardos, É. (2006). *Algorithm Design*. Addison-Wesley.
- [3] Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). *Introduction to Algorithms* (3rd ed.). MIT Press.